

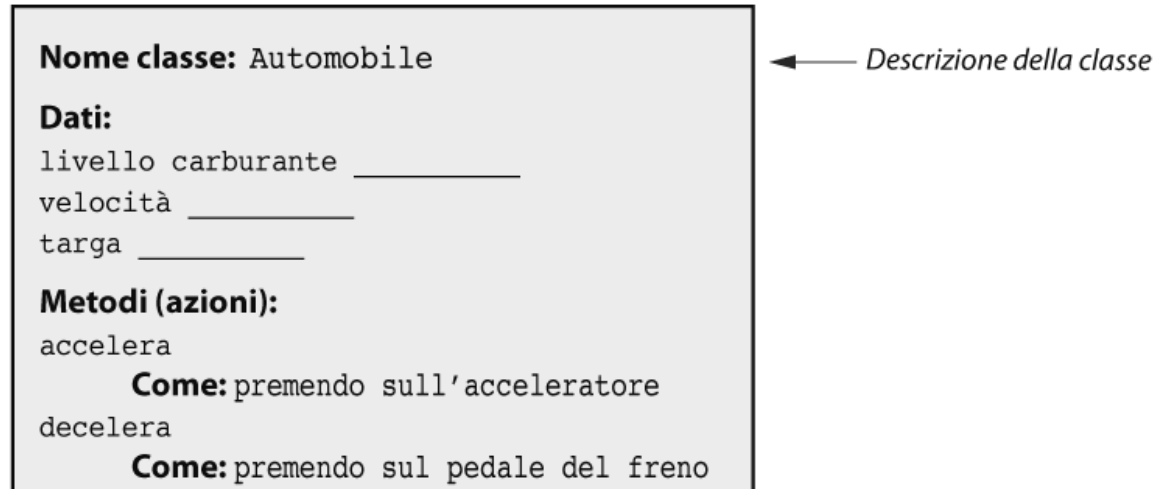
Programmazione in Java

Tutor-1-2:Classi e Oggetti in Java

Classi e Oggetti

- Il linguaggio Java è un linguaggio di programmazione strutturata ad oggetti
- Una classe è la definizione di un tipo di oggetto
- Gli oggetti interagiscono con altri oggetti
- Gli oggetti di un programma possono rappresentare:
 - Oggetti del mondo reale (macchine, animali, persone)
 - Astrazioni (forme , concetti, qualità,....)

Una classe in java



Prima istanziazione

Nome oggetto: autoDiLuca

livello carburante: 10 litri
velocità: 55 km all'ora
targa: DM 047 GH

Seconda istanziazione

Nome oggetto: autoDiMarco

livello carburante: 15 litri
velocità: 75 km all'ora
targa: LM 709 GH

Terza istanziazione

Nome oggetto: autoDiLaura

livello carburante: 25 litri
velocità: 0 km all'ora
targa: ZZ 999 ZZ

← *Oggetti che sono istanziazioni
della classe Automobile*

Classi e Oggetti

- Un programma Java consiste di **oggetti** che interagiscono tra loro attraverso i **metodi**.
- Un oggetto è una istanza di una **classe**
- Ovvero:

una **classe** è la definizione
di un tipo di oggetto

Files per le classi

- Ogni classe **Java** si mette solitamente in un file a sè che ha il nome della classe (e suffisso **.java**)
- Le classi possono essere compilate separatamente
- In genere le classi di uno stesso programma si tengono nella stessa cartella.

Definire una classe

- Ogni classe **Java** ha definiti:
 - **attributi** (variabili di istanza)
 - **azioni** (metodi di istanza)
- si mette solitamente in un file a sè che ha il nome della classe (e suffisso **.java**)
- La definizione di una classe NON specifica il valore degli attributi
- Le variabili di una classe sono accessibili a tutti i metodi della classe (non occorre passarle come parametri!)

Oggetti

- Un oggetto è una istanziatura di una classe ...
Nel programma diventa una variabile che ha come tipo la classe stessa.
- L'oggetto è dunque una variabile di tipo "complesso" che ha al suo interno vari tipi di variabili
- Un oggetto viene creato con l'operatore **new**.

Oggetti

- Sintassi:

```
<NClasse> <nOggetto> = new <NClasse> ();
```

Le variabili di istanza vengono inizializzate con i valori di default.

NOTA: le variabili locali ad un metodo invece non hanno una inizializzazione di default.

Metodi (di istanza)

- Possono essere void o restituire un valore.
- Accedono automaticamente a tutte le variabili di istanza
- All'interno della classe vengono chiamati direttamente (agiscono sull'oggetto implicito)
- All'esterno della classe vanno invocati usando un oggetto della classe

<nOggetto>.<nMetodo>()

La parola chiave **this**

- All'interno della classe un oggetto può anche essere esplicitato con il nome **this**
- Esempio:

```
this.name = keyboard.nextLine();
```
- La parola **this** corrisponde all'oggetto ricevente
- In genere si omette ma a volte è necessario utilizzarlo!

- Vediamo i programmi esempio:

Cane.java

CaneDemo.java

Oracolo.java

OracoloDemo.java

Dado.java

TestDado1.java

LancioDadi.java

Partita.java

Modificatori **public** e **private**

- Tipi specificati come **public**
 - Qualsiasi classe può accedere direttamente all'oggetto usando il suo nome
- Le classi in genere si definiscono come **public**
- Le variabili di istanza di solito sono non **public**
 - Si specifica **private**

Metodi set e get

- Quando le variabili di istanza sono private bisogna specificare dei metodi per accedere e per modificare il loro valore.
- I nomi usati sono tipicamente:
 - *getVariabile*
 - *setVariabile*

Vediamo direttamente sugli esempi...

Rettangolox.java

RettangoloxDemo.java

Variabili

- Tutte le variabili sono implementate come locazioni di memoria.
- Dati di *tipo primitivo* sono conservati nella locazione di memoria assegnata alla variabile
- Variabili di *tipo class* contengono l'indirizzo di memoria dell'oggetto corrispondente a quella variabile.

Variabili di tipo Class

- L'oggetto in sè non è conservato nella variabile
 - È da qualche parte nella memoria
 - La variabile contiene l'indirizzo di memoria dove è conservato
- L'indirizzo si chiama *riferimento (reference)* alla variabile
- Una variabile di tipo *reference type* contiene riferimenti (indirizzi) di memoria

Definire un Metodo `equals`

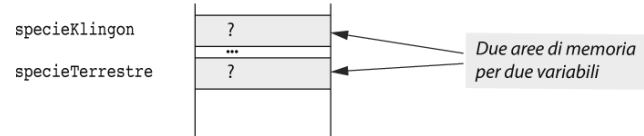
- Non possiamo usare `==` per confrontare due oggetti
- Per ogni classe dobbiamo anche scrivere un metodo che “realizza” il confronto (se serve nel programma)
- Osserviamo la differenza dei due metodi di confronto

....

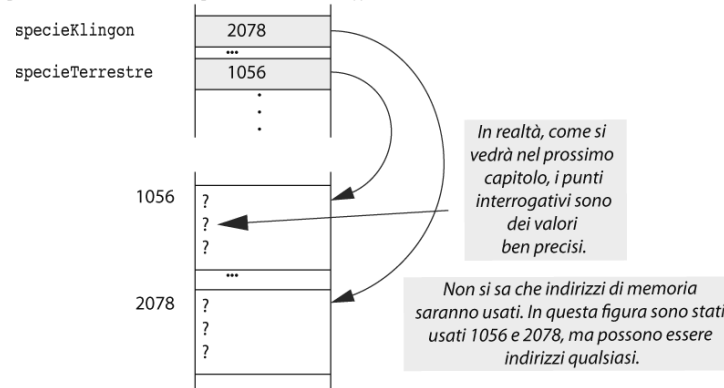
`==` versus `.equals ( )`

Comportamento di una variabile di tipo classe

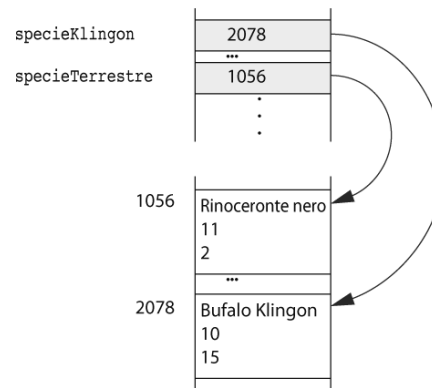
```
SpecieQuartaProva specieKlingon, specieTerrestre;
```



```
specieKlingon = new SpecieQuartaProva();
specieTerrestre= new SpecieQuartaProva();
```



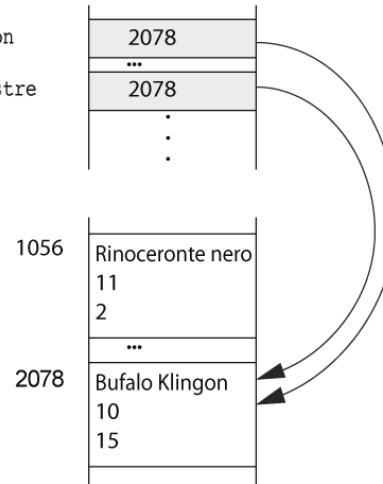
```
specieKlingon.setSpecie("Bufalo Klingon", 10, 15);
specieTerrestre.setSpecie("Rinoceronte nero", 11, 2);
```



Comportamento di una variabile di tipo classe

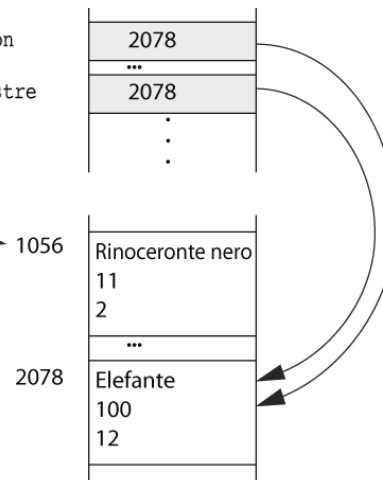
```
specieTerrestre = specieKlingon;
```

`specieKlingon` e `specieTerrestre`
puntano allo stesso oggetto
(sono due nomi dello stesso oggetto)



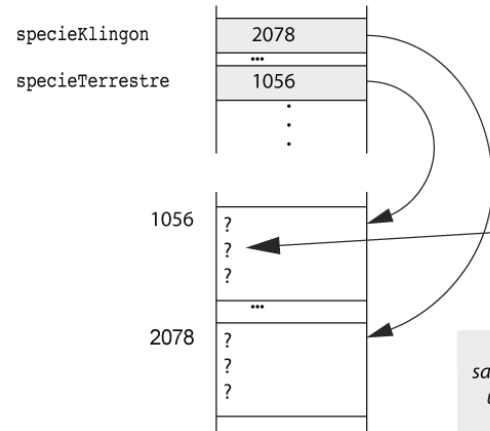
```
specieTerrestre.setSpecie ("Elefante", 100, 12);
```

*Questa non è più accessibile
dal programma.*



Il pericolo di usare == con gli oggetti

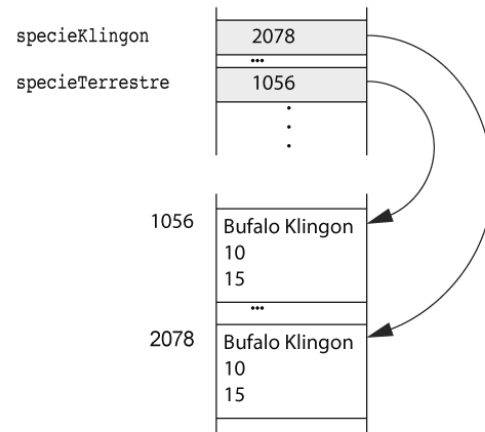
```
specieKlingon = new SpecieQuartaProva();  
specieTerrestre = new SpecieQuartaProva();
```



In realtà, come si vedrà nel prossimo capitolo, i punti interrogativi sono dei valori ben precisi.

Non si sa che indirizzi di memoria saranno usati. In questa figura sono stati usati 1056 e 2078, ma possono essere indirizzi qualsiasi.

```
specieKlingon.setSpecie("Bufalo Klingon", 10, 15);  
specieTerrestre.setSpecie("Bufalo Klingon", 10, 15);
```



```
if (specieKlingon == specieTerrestre)  
    System.out.println("Sono UGUALI");  
else  
    System.out.println("NON sono uguali");
```

L'output è NON sono uguali, poiché 2078 non è uguale a 1056.

Vediamo direttamente sugli esempi...

LanciaDadi.java

DadoPlus.java

LanciaDadi1.java

Costruttori

- Quando si crea una istanza di un oggetto con **new** viene chiamato un metodo speciale, chiamato **costruttore**, che:
 - Crea l'oggetto
 - Inizializza i valori delle variabili di istanza
- I costruttori hanno lo stesso nome della classe
- Sono definiti come altri metodi ma **SENZA** tipo di ritorno (neanche void)

Costruttori

- Possono avere dei parametri
 - Li uso per specificare dei valori iniziali
- Possono avere definizioni multiple (ognuna con diverso numero di parametri)

```
public class Animale {
    private String nome;
    private int    eta;    //in anni
    private double peso; //in Kg

    public Animale() {
        nome = "Nessun nome";
        eta = 0;
        peso = 0;
    }
    public Animale(String nomeIniziale, int etaIniziale,
                    double pesoIniziale){
        .....
    }
    public Animale(String nomeIniziale) {
        .....
    }
    public Animale(int etaIniziale) {
        .....
    }
    public Animale(double pesoIniziale) {
        .....
    }
}
```

Vediamo direttamente il file...

class Animale

- Nota i diversi costruttori:
 - Default
 - Con 3 parametri
 - Con 1 parametro **string**
 - Con 1 parametro **int**
 - Con 1 parametro **double**

Costruttori

- I metodi costruttori non si applicano agli oggetti ma si usano solo combinati con l'operatore **new**.
- Sono simili ai metodi **set** ma sono metodi speciali perchè si usano per CREARE gli oggetti oltre che inizializzarli

Costruttori

- Se non definiamo alcun costruttore, Java fornisce il costruttore di default (che è senza parametri)
- Se definiamo qualche costruttore, Java NON fornisce il costruttore di default.
- NOTA: Se definiamo qualche costruttore, è bene includere sempre anche quello senza parametri.

Costruttori

- Al loro interno posso mettere qualsiasi istruzione ma di solito si usano per inizializzare le variabili di istanza (uso metodi set per non ripetere codice!).
 - NOTA:Le variabili di istanza non inizializzate esplicitamente verranno inizializzate ai valori di default.
- ESEMPIO
`class Animale2`

Costruttori

- All'interno di un costruttore posso richiamare un altro costruttore. Uso la parola **this**.
Deve essere la prima azione eseguita.

- ESEMPIO

```
class Animale3
```

Overloading

- Si ha quando 2 o più metodi all'interno della stessa classe hanno lo stesso nome
- Java distingue i metodi dal numero e dai tipi dei parametri.
- Se una chiamata a un metodo non trova un corrispondente, Java prova a fare qualche conversione di tipo (es. Prova a convertire un **int** in **float**)
- Il numero e il tipo dei parametri si chiama la *signature* (firma) del metodo

Overloading e Conversioni di tipo

- Se una chiamata a un metodo non trova un corrispondente, Java prova a fare qualche conversione di tipo (es. Prova a convertire un `int` in `float`)
- ATTENZIONE: Overloading e conversioni di tipo possono creare un conflitto (vedi esempi): Java prova sempre a risolvere prima con l'overloading e poi con la conversione di tipo.
- Meglio evitare l'overloading quando non è necessario (per i costruttori è inevitabile)

Overloading e tipo restituito

- NON si può fare overloading di un metodo con la sola differenza del tipo restituito (deve esserci una diversa signature!)



Variabili static

- Le variabili static sono in comune a tutti gli oggetti della classe
- Le variabili dichiarate **static final** sono di fatto le costanti (il loro valore non può essere cambiato)
- Le variabili dichiarate **static** (senza **final**) possono essere cambiate
 - Esiste solo una istanza di tali variabili
 - Sono accessibili a tutti gli oggetti di quella classe

Variabili static

- Le ***variabili static*** sono anche chiamate ***variabili di classe*** in contrapposizione al termine ***variabili di istanza***.
- Non confondere le **variabili di classe** (=static) con le **variabili di tipo classe** (=oggetti)!
- Le variabili statiche e le variabili di istanza di una classe si chiamano anche *fields (campi)* della classe.
- Altro tipo sono le **variabili locali** (=definite all'interno dei metodi come supporto al calcolo).

[illegible]

Metodi static

- Sono metodi che non hanno relazione con alcun oggetto.
- Sono dichiarati in una classe e fuori dalla classe **si usano senza riferirli ad un oggetto** (ma preceduti da nome della classe!). Esempio i metodi della classe Math.
- Usati dentro la classe, **accedono automaticamente alle variabili static**.
- **NON accedono automaticamente alle variabili di istanza** (neanche usando this). Bisogna eventualmente passare l'oggetto come parametro.
- A volte è utile definire una classe di metodi static di supporto.

Vediamo degli esempi

```
class ContoBanca
```

```
class ContoBancaDemo
```

La costante `null`

- È un valore costante speciale che può essere assegnato ad una variabile di tipo classe (qualsiasi classe!)
- Esempio:

`ContoBanca mioconto = null;`

- Non è un oggetto: non si riferisce ad alcun indirizzo di memoria!

La costante `null`

- Si usa per verificare che il riferimento ad oggetto non è stato assegnato...
- Esempio:

```
if (mioconto == null){  
    mioconto = tuoConto;  
};
```

- Esempio

```
ContoBanca mioconto;  
mioconto.deposita(50.10);
```

Otengo errore *NullPointerException*

Array di oggetti

- Il tipo base di un array può essere una classe:

```
ContoBanca [ ] registroConti;
```

- Devo poi inizializzare l'array:

```
registroConti = new ContoBanca[10];
```

creo un vettore di oggetti ContoBanca che però non sono ancora inizializzati (anzi sono inizializzati a **null**)

- Devo poi inizializzare i singoli elementi con:

```
registroConti[i] = new ContoBanca();
```

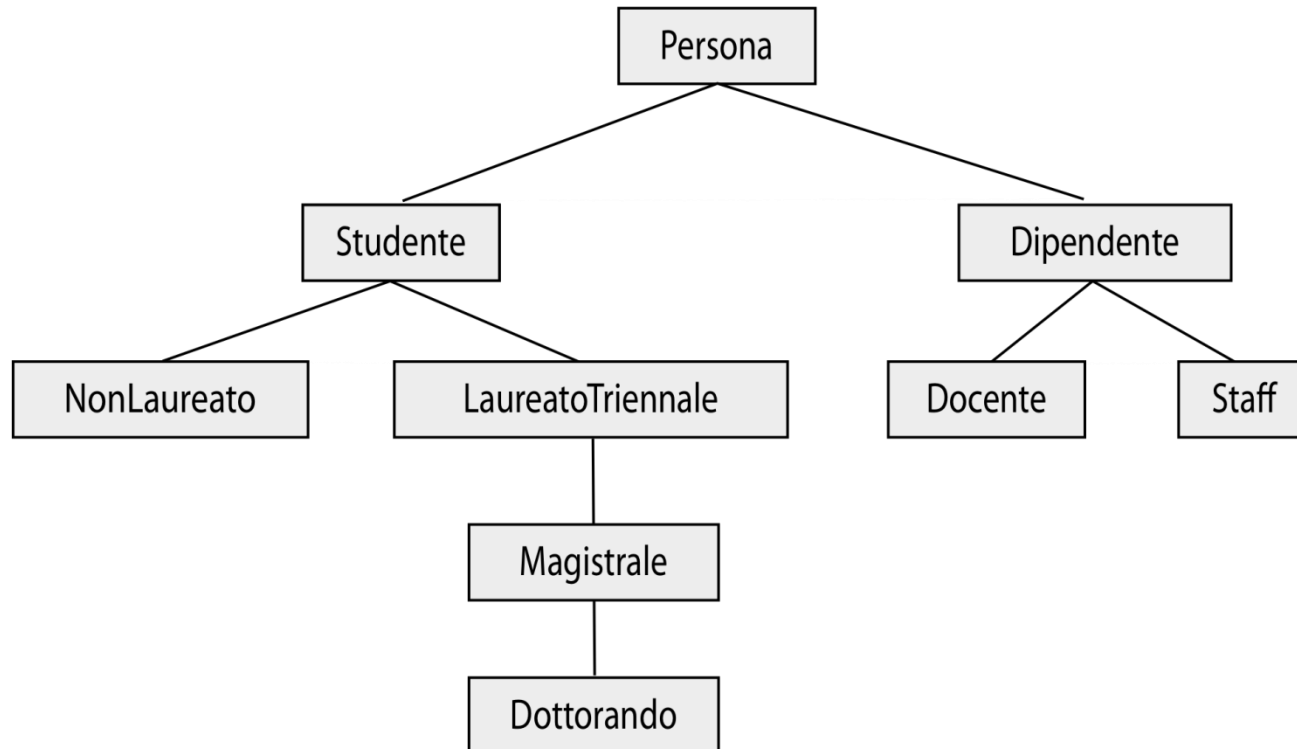
Vediamo l'esempio

```
class ContoBancaDemo2
```

Ereditarietà (Inheritance)

- Il programmatore definisce una classe generale
- In seguito definisce una classe più specifica
 - Aggiungo soltanto nuovi dettagli alla def generale
- La nuova classe eredita tutte le proprietà della classe iniziale, più generale
- Esempio: **class Persona**

Una gerarchia di classi



Classi Derivate

- Class **Persona** è usata come classe *base*
 - Detta anche *superclass*
- Dichiariamo una *derived* class **Studente**
 - Detta anche *subclass*
 - Eredita i metodi dalla superclasse

Overriding di Metodi

- In una sottoclasse posso avere un metodo con lo stesso nome della superclasse
- I metodi della sottoclasse con la stessa signature fanno overriding dei metodi della classe base
- I metodi definiti per overriding devono restituire valori dello stesso tipo

Overriding Versus Overloading

- Overriding
 - si trova in una subclass – nuovo metodo con la stessa signature
- Overloading
 - Nuovo metodo della stessa classe ma con diversa signature

Il modificatore **final**

- E' possibile specificare che un metodo non può essere “overridden” in una subclass
- Basta aggiungere il modificatore final
`public final void specialMethod()`
- Anche una intera classe può essere dichiarata **final**
 - In questo caso non può essere usata come classe base per derivare sottoclassi

Variabili e Metodi privati

- Una variabile privata nella classe base
 - E' ereditata nella sottoclasse
 - Può essere manipolata SOLO attraverso i metodi set e get pubblici
 - NOTA: se non fosse così si potrebbe usare la def di una sottoclasse per accedere direttamente ai membri privati!
- Similmente i metodi privati di una superclasse non sono ereditati da una sottoclasse.
 - (Se la superclasse ha un metodo pubblico che usa il metodo privato, ok)

Costruttori nelle Classi derivate

- Una classe derivata NON eredita i costruttori dalla classe base
 - Il costruttore di una sottoclasse DEVE invocare il costruttore della classe base
- Uso la parola riservata **super** (ruolo simile a **this**)
 - Deve essere la prima azione del costruttore
- Se non invoco il costruttore della classe base viene invocato automaticamente il costruttore di default (se non esiste: ERRORE!)

Il metodo **this**

- E' anche possibile usare **this** per chiamare un qualsiasi costruttore della stessa classe

```
public Person()  
{  
    this("No name yet");  
}
```

- In contrasto con uso **super** che chiama un costruttore della classe base
- In un costruttore non si può usare sia **this** che **super**

Chiamare un metodo che è stato “Overridden”

- La parola **super** può essere usata per chiamare un metodo nella nuova definizione data per overriding

```
public void writeOutput()  
{  
    super.writeOutput(); //Display the name  
    System.out.println("Student Number: " + studentNumber);  
}
```

- Chiama il metodo con lo stesso nome nella classe base

Classi derivate da classi derivate....

```
class NonLaureato extends Studente
```

- Contiene tutti i membri e metodi di
 - **Persona**
 - **Studente**
- Riutilizzo il codice di entrambe le superclassi

Compatibilità di tipi

- Nella gerarchia delle classi
 - Ogni **NonLaureato** è anche **Studente**
 - Ogni **Studente** è anche **Persona**
- Un oggetto di una classe derivata può essere utilizzato al posto di un oggetto della classe base
 - NON è un typecasting

Studente s = new Studente (); OK

Persona p = s; OK

Persona p = new Studente (); OK

Studente s = new Persona (); NO

La Classe **Object**

- Java ha una classe che è l'antenata di tutte le classi (il progenitore?): la classe **Object**
- Posso scrivere un metodo con parametro di tipo **Object**
 - Il parametro attuale nella chiamata di tale metodo può essere un oggetto di qualsiasi classe
- Esempio: il metodo
println(Object unOggetto)

La Classe **Object**

- La Classe Object ha alcuni metodi (che sono quindi ereditati da tutte le classi Java)
 - **equals** :per confrontare due oggetti
 - **toString** :viene chiamato quando si usa l'istruzione **println(theObject)**
- Conviene sempre ridefinire il proprio **toString** e il proprio **equals**

Ridefinire metodo `toString`

- `toString` restituisce tutti i dati di un oggetto “impacchettati” in una stringa
- l'oggetto `system.out` ha vari metodi `println` (definiti per overloading).

```
public void println(Object unOggetto) {  
    System.out.println(unOggetto.toString());  
}
```

- ESEMPIO `class Persona2`

ESERCITAZIONE

Classe Persona

- Nome (String)
- Anno nascita (int)
- Costruttori, metodi set/get/ toString

Classe Insegnamento

- annoDiCorso (int) (valore tra 1 e 3)
- nomeCorso (String)
- Costruttori, metodi set/get/ toString

Classe Docente (estende Persona)

- materia1, materia2 (Insegnamento)
- Costruttori, metodi set/get/ toString

Classe Esame estende Insegnamento

- voto (int) (valore ≥ 18 e ≤ 30)
- lode (boolean) (può essere true solo se voto = 30)
- Costruttori, metodi set/get/ toString

Classe Studente (estende Persona)

- Campi:
 - Matricola (String)
 - ListaEsami (array di Esame)
- Metodi:
 - Costruttori, metodi set/get/ toString
 - Media su tutti gli esami
 - Media su esami singolo annoDiCorso
 - (fare overloading)

Main

- Definire un oggetto docente e inizializzarlo con dati da tastiera (o direttamente nel main).
- Stampare poi tutti i dati.
- Definire un oggetto studente e inizializzarlo con dati da tastiera (o direttamente nel main).
- Stampare poi tutti i dati.
- Verificare se lo studente ha superato uno degli esami insegnati dal docente e in caso affermativo scrivere il voto ottenuto.
- Scrivere anche se tale voto è maggiore o minore della media dei voti degli esami di quello stesso anno.

Esercizio 2

- Scrivere una **classe studente** : ogni studente ha un nome, un numero di matricola e un vettore esami1anno di 5 elementi che conterranno i voti conseguiti. Se un esame non è stato superato, il valore corrispondente sarà 0. Esiste un metodo media che calcola la media degli esami. Attenzione: la media deve essere fatta tra gli esami effettivamente conseguiti (cioè non deve considerare quelli con valore 0). La classe è completa di costruttori, metodi set e get, metodo di stampa e metodo equals. La matricola viene generata in modo sequenziale a partire dal numero 22222 per ogni nuovo studente (usare variabile static nella classe per contare le istanze).
- Scrivere poi un programma (classe con il **main**) che inizializza un array di 10 studenti chiedendo nome all'utente e generando random 5 voti (valori tra 18 e 30 oppure 0 se l'esame non è stato sostenuto) per gli esami e poi calcola chi è lo studente che ha conseguito il voto più alto in qualche esame e chi è quello con la media più alta (a parità di media il programma dovrà segnalare lo studente che ha fatto più esami).

Esercizio 2 (cont.)

- Scrivere poi un programma (classe con il **main**) che inizializza un array di 10 studenti chiedendo nome all'utente e generando random 5 voti (valori tra 18 e 30 oppure 0 se l'esame non è stato sostenuto) per gli esami e poi calcola chi è lo studente che ha conseguito il voto più alto in qualche esame e chi è quello con la media più alta (a parità di media il programma dovrà segnalare lo studente che ha fatto più esami). Il programma dovrà riportare anche il nome dell'insegnamento dove tale studente ha il voto più alto e il nome del docente corrispondente

